Blue-Green Deployments

switch in seconds, either direction

Watch the 4-min video

In one line: run two identical production environments, deploy and test the new version on the idle one, then move all traffic at once; switching back is just as fast, as long as both versions can run against the same database (expand and contract) and you pay for two environments.

1 Twenty minutes to roll back

Friday evening, a deploy goes out. Errors spike, and rolling back takes twenty minutes while customers watch, because rollback means another full deployment (lesson 73). What if switching versions took one second, in either direction? That is the promise of blue-green deployment.

2 Two environments, one switch

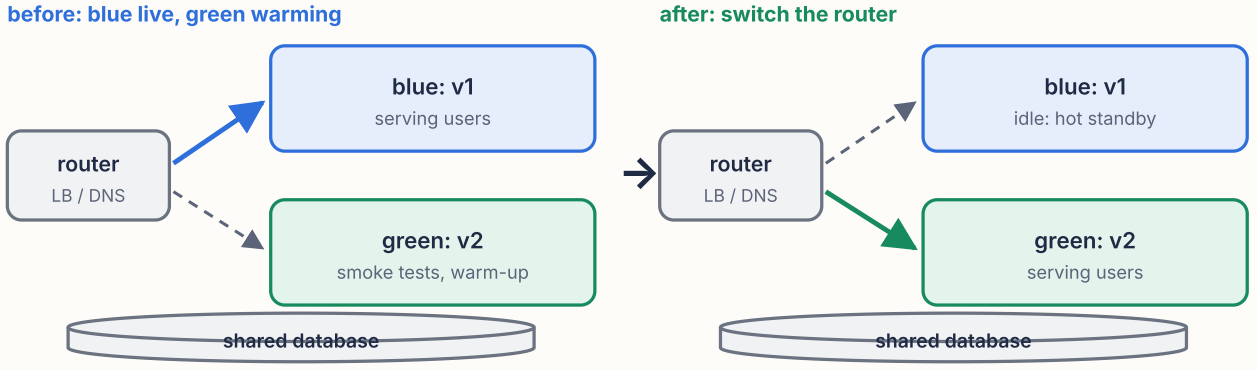


Figure 1. Green is production in every way except one: no customers yet. The cutover is a routing change, so rollback is the same change in reverse.

Blue serves all live traffic. **Green** gets the new version, fully deployed and warmed up, with no user traffic. Test green with smoke tests against real production dependencies. Then the router (load balancer, ingress, or DNS) moves every request to green at once. Blue stays running, untouched, as a hot standby. If green misbehaves, switch back in seconds. Prefer switching at a load balancer: DNS changes are slowed by TTLs and clients that cache.

3 The hard part: the database

Both colours usually share one database, so during and after the switch, old and new code must work with the same schema. Use **expand and contract**: expand the schema in a backward-compatible way while blue still runs, switch, and contract (drop old columns) only in a later release, once rollback to blue is no longer needed.



Figure 2. Splitting a name column across two releases. A destructive migration in the same release as the switch would make instant rollback impossible.

4 Other state

- **Draining:** long requests, sessions and WebSocket connections on blue need time to finish or reconnect; keep sessions in a shared store.
- **Background workers:** queue consumers and cron jobs must not run in both colours at once, or jobs run twice.
- **Caches:** warm green before cutover, or the first minute is slow and hammers the database.
- **Side effects:** smoke tests on green hit real dependencies: use test accounts, not real payments.

5 Comparing strategies

	Rolling (lesson 73)	Blue-green	Canary (lesson 75)
Switch	gradual, by instance	all at once	gradual, by traffic %
Rollback	minutes (another rollout)	seconds (route back)	seconds (weight to 0)
Extra cost	a few surge pods	a second full environment	small canary set + tooling
Blast radius	grows during rollout	everyone, at once	small slice first
Version skew	yes, during rollout	shared DB only	yes, during analysis

TRADE-OFFS

- **Cost:** two full environments can double the bill; many teams keep green small and scale it up only during a release, or create it on demand.
- **All-at-once exposure:** every user hits green together; problems that need real load show up for everyone.
- **Fastest rollback** of any strategy, and a full production-like test before any user sees the change.
- **Schema discipline** is mandatory; without it, the instant rollback is an illusion.

WORKED EXAMPLE: AZURE APP SERVICE DEPLOYMENT SLOTS

The **production** slot serves users; a **staging** slot receives the new build. Before a swap, App Service warms up the staging instances by sending requests to them. Then `az webapp deployment slot swap -g shop -n shop-api --slot staging --target-slot production` exchanges the slots: the new build is production in seconds, with no cold start. Settings marked as **deployment slot settings** (e.g. a staging database connection string) stay with their slot. If anything looks wrong, swap again and the previous version is back. (Slots need the Standard tier or higher.)

6 Key terms

Blue / green: the live and the idle production environments

Hot standby: old environment kept running for instant rollback

Deployment slot: App Service's built-in blue-green environment

Cutover: moving all traffic to the new environment at once

Expand and contract: backward-compatible schema change over releases

Slot setting: config that stays with its slot on swap

7 Interview questions

Q1. How do you handle database schema changes with blue-green? BASIC

Expand and contract: make every change backward compatible so both versions run against the same database, switch, and remove old columns only in a later release once rollback is no longer needed.

Q2. What's the main trade-off of blue-green? BASIC

An instant, all-at-once switch and the fastest rollback, in exchange for running two full environments for a while.

Q3. After cutover, every order is processed twice. Why? SCENARIO

Queue consumers or scheduled jobs are running in both blue and green. Run workers in only the live colour, or use leader election and idempotent processing.

Q4. Why might switching via DNS give a slow, messy cutover? SCENARIO

Resolvers and clients cache records for the TTL or longer, so traffic drains to green gradually and unpredictably; a load-balancer switch is immediate.

Q5. When does blue-green rollback stop being instant? TRICKY

When green has made changes blue can't handle: a dropped column, a new message format, data written in a new shape.

Q6. Blue-green or canary for a risky new algorithm? TRICKY

Canary: blue-green exposes all users at once, while a canary limits the blast radius and compares metrics on real traffic first.

Next: SD-N-017 · Canary Deployments backendroadmap.com